Smart Retail Demand Forecasting — Academic Project Roadmap

Refined Proposal + Functional Requirements + Execution Workflow

Version: 1.0 | **Date:** March 2026 | **Project Type:** Data Science Semester Project

---

SECTION 1 — Refined Project Definition

1.1 Problem Statement

Retail chains managing thousands of SKUs face a dual failure mode: overstocking generates capital inefficiency and spoilage risk (particularly in perishable categories), while understocking causes revenue leakage and customer churn. Existing demand planning tools apply univariate time-series models per SKU and ignore three critical dynamics:

1. **Promotional lift** — transient demand spikes caused by discounts, bundles, or end-cap placements.
2. **Cannibalization** — one product's promotion suppresses adjacent SKU demand within the same category.
3. **Cross-SKU interaction** — correlated demand patterns across product lines driven by seasonality, consumer habits, or complementarity.

This project builds a **multi-family retail demand forecasting system** using data from Corporación Favorita (Ecuador's largest grocery chain) that produces accurate daily sales predictions while quantifying promotional impact and identifying cannibalization pairs across 33 product families and 54 stores.

1.2 ML Problem Type

Dimension	Specification
Primary task	Supervised regression (continuous sales volume prediction)
Secondary task	Unsupervised clustering (cannibalization group detection)
Problem subtype	Multi-step time-series forecasting (16-day-ahead horizon, per competition spec)
Scope	Multi-family, multi-store panel regression (54 stores × 33 product families)

1.3 Target Variable

- `sales` — total unit sales for a product family at a specific store on a given date. Fractional values are valid (e.g., 1.5 kg of cheese).
- One row = one `(store_nbr, family, date)` triple.
- Prediction horizon: **16 days ahead** (Aug 16–31, 2017), matching the competition test window. For academic purposes, this is treated as a hold-out evaluation period.

1.4 Input Features (Expected)

Feature Category	Specific Features (Favorita columns)
Temporal	`day_of_week`, `day_of_month`, `week_of_year`, `month`, `quarter`, `year`, `is_weekend`
Holiday features (`holidays_events.csv`)	`is_national_holiday`, `is_regional_holiday`, `is_local_holiday`, `holiday_type`, `days_to_holiday`, `is_transferred`
Lag features	`sales_lag_1d`, `sales_lag_7d`, `sales_lag_14d`, `sales_lag_365d`
Rolling statistics	`rolling_mean_7d`, `rolling_mean_14d`, `rolling_mean_28d`, `rolling_std_7d`
Promotional	`onpromotion` (count of items on promotion per family/store/day)
Store metadata (`stores.csv`)	`store_type` (A–E), `store_cluster` (1–17), `city`, `state`
Macroeconomic (`oil.csv`)	`dcoilwtico` (daily WTI crude oil price — direct economic driver for Ecuador)
Transaction volume (`transactions.csv`)	`transactions` (daily store-level foot traffic — leading indicator of sales)
Cross-family (cannibalization)	`family_avg_onpromotion`, `competitor_family_sales_lag_1d` (top-3 correlated families per store)

1.5 Assumptions

1. **Dataset**: The **Corporación Favorita Grocery Sales Forecasting** dataset (Kaggle competition: `store-sales-time-series-forecasting`) is used. It contains 54 stores, 33 product families, and daily sales from **January 1, 2013 to August 31, 2017** (~3 million rows). Six CSV files are provided: `train.csv`, `test.csv`, `stores.csv`, `oil.csv`, `holidays_events.csv`, `transactions.csv`.
2. **Granularity**: Forecasting is performed at the `(store_nbr, family, date)` level (daily). Where "SKU" is referenced in the proposal, it is interpreted as a `(store, product_family)` pair.
3. **Cannibalization proxy**: True basket-level cannibalization cannot be computed from this dataset. Cross-elasticity between product families will be estimated using lagged residual sales of correlated families during `onpromotion > 0` periods as a proxy.
4. **Oil price imputation**: `dcoilwtico` has ~44 missing values (weekends/holidays). Strategy: forward-fill, then backward-fill. Oil price is a documented economic driver for Ecuador and must be included.
5. **Evaluation period**: The official competition test period is Aug 16–31, 2017 (16 days). For academic grading, this same window is used as the hold-out test set. Training uses all data before Aug 16, 2017.
6. **From-scratch gradient descent**: A Linear Regression with Gradient Descent (GD) is implemented manually (NumPy only) as a baseline interpretability model, satisfying the proposal deliverable requirement.

1.6 Success Metrics

Metric	Primary Use	Target Threshold
RMSLE (Root Mean Squared Log Error)	**Official competition metric** — primary	< 0.45
RMSE (Root Mean Squared Error)	Raw-scale interpretability	Minimize
MAPE (Mean Absolute Percentage Error)	Scale-invariant comparison across families	< 15%
R^2	Variance explained	> 0.80

> **Why RMSLE?** The target `sales` is right-skewed and includes zero values. RMSLE penalizes under-prediction more than over-prediction, which is appropriate for retail (stockout cost > overstock cost). Compute as: `RMSLE = sqrt(mean((log1p(y_pred) - log1p(y_true))^2))`.

> **Cannibalization sub-metric**: Pearson cross-correlation coefficient between `onpromotion`-activated family residual sales and adjacent family residual sales. A negative correlation (< −0.3) during promotion windows is considered evidence of cannibalization.

SECTION 2 — Functional Requirements (8 Requirements)

FR-01: Data Ingestion and Validation Module

Description:
The system must load all six Favorita CSV files (`train.csv`, `test.csv`, `stores.csv`, `oil.csv`, `holidays_events.csv`, `transactions.csv`), merge them on appropriate keys (`store_nbr` + `date`), and perform automated schema validation: column existence checks, data type verification, range checks (e.g., `sales` must be ≥ 0), and null-count reporting per column.

Why it is necessary:
Without a validated, merged base dataset no downstream processing is possible. The Favorita dataset requires multi-table joins across 6 files with different key structures — a join error silently corrupts features like `dcoilwtico` or `transactions` for entire stores.

Measurable Acceptance Criteria:

- All six source files are successfully merged into a single panel DataFrame; merge diagnostics (left-join null counts) are printed.
- A validation report is auto-generated listing: shape, dtypes, null counts per column, and any `sales < 0` violations.
- The merged DataFrame contains exactly `54 stores × 33 families × training_days` rows (minus any missing combinations); discrepancy is flagged.
- Execution time does not exceed 60 seconds on a standard laptop (dataset is ~3M rows).

FR-02: Data Preprocessing Pipeline

Description:
The pipeline must handle: (a) forward-fill imputation of missing `dcoilwtico` values (weekends/public holidays

have no oil price recorded), (b) classification of holidays in `holidays_events.csv` — distinguish `transferred=True` holidays (treat as workday) from active holidays, (c) outlier detection using IQR per `(store_nbr, family)` group, and (d) temporal ordering of all records by `(store_nbr, family, date)`.

Why it is necessary:
The Favorita `oil.csv` has ~44 missing rows (no trading on weekends). `holidays_events.csv` contains `transferred=True` rows that mark days officially designated as holidays but observed on a different date — treating these as holidays introduces incorrect feature values. Unhandled missing oil prices will propagate NaN through all downstream features.

Measurable Acceptance Criteria:

- After preprocessing, `dcoilwtico` has 0% null values; imputation method is documented.
- Transferred holidays are correctly excluded from the `is_holiday` flag (Boolean logic verified on ≥ 2 known transfer dates).
- All records are sorted chronologically within each `(store_nbr, family)` group.
- The final preprocessed DataFrame has a reproducible row count logged to file.

FR-03: Feature Engineering Module

Description:
Construct the following feature groups programmatically:

1. **Temporal features**: `day_of_week`, `day_of_month`, `week_of_year`, `month`, `quarter`, `year`, `is_weekend`.
2. **Holiday features**: `is_national_holiday`, `is_regional_holiday`, `is_local_holiday`, `holiday_type` (encoded), `days_to_nearest_holiday`, `is_transferred` (binary).
3. **Lag features**: Sales lag at 1, 7, 14, and 365 days per `(store_nbr, family)` group.
4. **Rolling statistics**: 7-day, 14-day, and 28-day rolling mean and standard deviation per `(store_nbr, family)` group.
5. **Promotional feature**: `onpromotion` (raw count from `train.csv`), `onpromotion_lag_1d`, `onpromotion_rolling_7d`.
6. **Macroeconomic**: `dcoilwtico` (oil price), `dcoilwtico_lag_7d`, `dcoilwtico_rolling_28d`.
7. **Transaction volume**: `transactions` (from `transactions.csv`), `transactions_lag_7d`.
8. **Store metadata**: `store_type` (label-encoded A–E), `store_cluster` (1–17), `city`, `state` (target-encoded by mean sales).
9. **Cannibalization proxy features**: For each `(store, family, date)`, add the lagged mean sales of the top-3 most correlated other families in the same store.

Why it is necessary:
Raw temporal data provides no signal to tree-based models (which cannot extrapolate). Lag and rolling features encode the autoregressive dynamics essential for time-series forecasting. Without these, models degrade to simple cross-sectional regressors.

Measurable Acceptance Criteria:

- All 9 feature groups are present in the final feature matrix.
- Lag features contain NaN only for the first `k` rows per group (where `k` = lag period); these rows are dropped before model training.

- Feature engineering is implemented as a reusable Python class/function accepting a DataFrame and returning an augmented DataFrame.
- Unit test: confirm `sales_lag_7d` of a constructed test series equals the known ground truth.

FR-04: Exploratory Data Analysis (EDA) Module

Description:
The EDA module must produce the following documented outputs: (a) time-series decomposition plots (trend, seasonality, residual) for at least 3 `(store_nbr, family)` combinations using `statsmodels.seasonal_decompose`, (b) a correlation heatmap of numerical features, (c) distribution plots for `sales` and `onpromotion`, (d) holiday lift analysis — bar chart comparing mean sales across national, regional, local, and non-holiday days, (e) oil price vs. aggregate sales scatter plot over time, and (f) cross-family correlation matrix to identify cannibalization candidate pairs.

Why it is necessary:
EDA is the primary mechanism for validating assumptions about the data. Without it, feature engineering decisions (e.g., lag window size, seasonal period) are arbitrary and undefended in academic evaluation.

Measurable Acceptance Criteria:

- Minimum 10 publication-quality figures saved to `/outputs/eda/` directory.
- Each figure has titles, axis labels, and source annotations.
- EDA notebook/script outputs at least 3 documented insights (written as markdown cells) that directly inform feature engineering or modeling decisions.
- Time-series decomposition uses `period=365` (daily data, annual seasonality) and this is explicitly documented.

FR-05: Model Training Pipeline (Baseline + Advanced)

Description:
Implement and train the following models in sequence:

1. **Baseline 1**: Seasonal Naive (persistence model — predict last year's same-week value).
2. **Baseline 2**: Linear Regression with Gradient Descent (implemented from scratch using NumPy; no `sklearn.LinearRegression`).
3. **Advanced 1**: XGBoost Regressor.
4. **Advanced 2**: LightGBM Regressor.
5. *(Optional)* **Advanced 3**: Prophet (Facebook), if time permits.

All models must use **temporal train/test split**: training on data from Jan 1, 2013 to Aug 15, 2017; test set is Aug 16–31, 2017 (the official 16-day competition window). No random shuffling of time-series data.

Why it is necessary:
A baseline model is required to establish a performance floor. Without it, advanced model improvements cannot be quantified or defended. The from-scratch GD implementation satisfies the stated project deliverable requirement.

Measurable Acceptance Criteria:

- All models produce predictions on the held-out 16-day test set.
- Training and inference times are logged per model.
- The from-scratch GD implementation converges (loss decreases monotonically or plateaus) within 1000 iterations; convergence curve is plotted.
- Zero data leakage: confirmed via a documented check that no test-set dates (Aug 16–31, 2017) appear in any training feature window.

FR-06: Hyperparameter Tuning Module

Description:
Tune XGBoost and LightGBM using one of the following strategies: (a) `GridSearchCV` with `TimeSeriesSplit` (5 folds), or (b) Bayesian optimization using `Optuna`. The following hyperparameters must be included in the search space:

- XGBoost: `n_estimators`, `max_depth`, `learning_rate`, `subsample`, `colsample_bytree`.
- LightGBM: `num_leaves`, `max_depth`, `learning_rate`, `min_child_samples`, `feature_fraction`.

Why it is necessary:
Default hyperparameters result in suboptimal models. More critically, using standard `KFold` cross-validation on time-series data causes data leakage (future data is used to predict past). `TimeSeriesSplit` enforces temporal ordering.

Measurable Acceptance Criteria:

- Tuning uses `TimeSeriesSplit` with `n_splits ≥ 3`; this is explicitly configured and logged.
- Best hyperparameters are saved to a JSON file (e.g., `best_params_xgb.json`).
- Post-tuning RMSE improves by ≥ 3% relative to pre-tuning RMSE on the validation fold (or the result and explanation are explicitly documented if improvement is marginal).
- Total tuning wall-clock time is logged.

FR-07: Model Evaluation Framework

Description:
A centralized evaluation module must compute the following for every trained model: RMSE, MAPE, R^2, and MAE on the held-out test set. Additionally: (a) plot actual vs. predicted sales for the top-5 store-department combinations by sales volume, (b) compute residual plots and check for systematic bias (mean residual significantly ≠ 0 indicates model misspecification), and (c) perform SHAP value analysis on the best-performing tree model to identify the top-10 most important features.

Why it is necessary:
A single metric is insufficient for model assessment. Residual analysis detects heteroscedasticity and structural bias. SHAP analysis provides feature importances that are consistent (unlike built-in tree importance) and are required for the cannibalization and promotion-effect interpretation deliverables.

Measurable Acceptance Criteria:

- All 4 metrics are reported in a comparative table (model vs. metric).
- Actual vs. predicted plots are generated for ≥ 5 store-dept series.
- SHAP summary plot identifies at least 1 promotional feature and 1 temporal feature in the top-5.

- Residual mean is reported per model; if `|mean residual| > 5% of mean actual sales`, a bias explanation is documented.

FR-08: Cannibalization & Promotional Impact Analysis Module

Description:
This module must:

1. Compute a **cross-department correlation matrix** on residual sales (sales net of trend and seasonality), targeting periods with active markdowns.
2. Identify **cannibalization candidate pairs** using a threshold of Pearson $r < -0.35$ during promotional periods.
3. Quantify **promotional lift** for each markdown event: `lift = (actual_sales - counterfactual_baseline) / counterfactual_baseline`, where counterfactual baseline is the rolling 4-week pre-promo average.
4. Produce a scatter plot of markdown amount vs. promotional lift per department.

Why it is necessary:
This is the highest-differentiation deliverable in the proposal. It elevates the project from a standard regression exercise to a business-insight-generating analysis. Without it, the project fails its own stated objective of "quantifying cannibalization."

Measurable Acceptance Criteria:

- At least 3 statistically significant cannibalization pairs are identified and documented (with Pearson r and p-value reported).
- Promotional lift is computed for ≥ 10 distinct markdown events.
- A final cannibalization report (3–5 pages) is saved as a PDF or markdown document with visual evidence.
- All findings can be reproduced by re-running the module script with the same random seed.

FR-09: Model Persistence and Reporting Module

Description:
(a) The best-performing model must be saved using `joblib` (or `pickle` for the GD model) to a `/models/` directory with a filename convention: `model_name_YYYYMMDD_vX.pkl`. (b) A final project report template (Jupyter Notebook + exported HTML/PDF) must be structured with: executive summary, methodology, results table, EDA highlights, model comparison, cannibalization findings, limitations, and future work sections.

Why it is necessary:
Model persistence is mandatory for reproducibility — a core academic requirement. Without a structured report, findings exist only in scattered notebook cells, which is insufficient for academic submission.

Measurable Acceptance Criteria:

- Saved model can be reloaded and produces identical predictions (bit-for-bit) on the test set without retraining.
- Final report template has all 8 required sections defined.
- Report notebook executes end-to-end (Kernel → Restart & Run All) without errors.

- Exported HTML/PDF report file size is ≤ 20MB.

SECTION 3 — Complete Data Science Workflow (Step-by-Step)

PHASE 1 — Problem Framing

Inputs: Project proposal PDF, domain knowledge, dataset documentation.
Outputs: Signed-off problem statement, metric definitions, feasibility assessment.

Steps:

1. Formalize the forecasting objective as: "*Minimize RMSLE on daily sales prediction for the held-out Aug 16–31, 2017 test period across all (store_nbr, family) pairs.*"
2. Confirm the prediction horizon: **16 days ahead** (fixed by competition). For academic purposes, also report RMSE and MAPE.
3. Define success thresholds: RMSLE < 0.45; MAPE < 15%; R^2 > 0.80.
4. Define constraints:
   - No data beyond the Favorita competition dataset is used unless explicitly noted.
   - Temporal integrity must be preserved: no shuffling of time-series.
   - From-scratch gradient descent implementation is mandatory (as per deliverables).
5. Document all assumptions (see Section 1.5).
6. Confirm dataset availability: download Favorita dataset from Kaggle and verify file hashes.

Common Mistakes to Avoid:

- ❌ Treating this as a classification problem (sales is continuous — use regression metrics).
- ❌ Defining MAPE as the only metric (MAPE is undefined when actual = 0; use RMSLE as primary — `log1p` handles zeros).
- ❌ Failing to account for the temporal nature — this is NOT a standard i.i.d. regression problem.

PHASE 2 — Data Acquisition

Inputs: Kaggle API key or manual download.
Outputs: Raw data files stored in `/data/raw/`, integrity-verified.

Steps:

1. Register for Kaggle API and download the dataset:

   ```
   kaggle competitions download -c store-sales-time-series-forecasting
   unzip store-sales-time-series-forecasting.zip -d data/raw/
   ```

   Files: `train.csv` (~3M rows), `test.csv` (28,512 rows), `stores.csv`, `oil.csv`, `holidays_events.csv`, `transactions.csv`, `sample_submission.csv`.
2. Verify file sizes match expected (e.g., `train.csv` ≈ 112MB uncompressed, 54 stores × 33 families).
3. Document dataset provenance:

    - Source: Kaggle — Store Sales Time Series Forecasting Competition (2021)
    - Data origin: Corporación Favorita, Ecuador
    - License: Kaggle competition rules (for academic/educational use)
    - Citation: *Corporación Favorita. (2021). Store Sales — Time Series Forecasting. Kaggle. https://kaggle.com/competitions/store-sales-time-series-forecasting*
4. Load all files into Pandas DataFrames and print `.info()` and `.describe()` for each.
5. Record: row counts per file, column types, memory usage, date range (`2013-01-01` to `2017-08-31`), store count (54), family count (33).

Common Mistakes to Avoid:

- ❌ Using the Kaggle test set (`test.csv`) for model evaluation — it has no ground truth `sales` labels. Create your own hold-out split from `train.csv` using the final 16 days.
- ❌ Not logging which version of the dataset was used. Store the download timestamp and save SHA-256 hashes of all input files to `data/checksums.txt`.

PHASE 3 — Data Preprocessing

Inputs: Raw merged DataFrame.
Outputs: Cleaned DataFrame saved to `/data/processed/cleaned_data.parquet`.

Steps:

3.1 — Merge all source tables:

```
train = pd.read_csv('data/raw/train.csv', parse_dates=['date'])
stores = pd.read_csv('data/raw/stores.csv')
oil    = pd.read_csv('data/raw/oil.csv', parse_dates=['date'])
holidays = pd.read_csv('data/raw/holidays_events.csv', parse_dates=['date'])
txns   = pd.read_csv('data/raw/transactions.csv', parse_dates=['date'])

df = train.merge(stores, on='store_nbr', how='left')
df = df.merge(oil, on='date', how='left')
df = df.merge(txns, on=['store_nbr', 'date'], how='left')
# Holidays are processed separately and joined as feature flags (see 3.2)
```

3.2 — Handle missing oil prices and engineer holiday flags:

- `dcoilwtico`: ~44 missing values. Strategy: **forward-fill, then backward-fill** within the date-sorted series. Document count of filled rows.
- `holidays_events.csv` processing:

```
# Keep only non-transferred holidays as actual holidays
active_holidays = holidays[holidays['transferred'] == False].copy()
# Create separate flags per locale
national = active_holidays[active_holidays['locale'] == 'National']
[['date','type']]
```

```
df['is_national_holiday'] = df['date'].isin(national['date']).astype(int)
# Repeat for Regional (match locale_name to state) and Local (match to city)
```

- `transactions`: ~2% missing (some store-days with no records). Forward-fill within `store_nbr` group.

3.3 — Handle zero `sales`:

- Zero sales are **valid** in the Favorita dataset (a store may genuinely sell zero units of a product family on a given day). Do NOT impute or remove zeros.
- Log the count and percentage of zero-sale rows per family as a data quality note.

3.4 — Outlier handling:

- Compute IQR per `(store_nbr, family)` group.
- Flag rows with `sales > Q3 + 3*IQR` as outliers (extreme spike days, e.g., during earthquake 2016). Do NOT remove; add `is_outlier` binary flag for model awareness.
- Note: The April 2016 Ecuador earthquake caused documented sales anomalies — flag this date range (`2016-04-16` to `2016-05-15`) explicitly.

3.5 — Data type enforcement:

- Convert `date` to `datetime64`.
- Label-encode `type` (store type: A–E → 0–4) and `cluster` (already integer).
- Target-encode `city` and `state` using leave-one-out mean encoding on training data only.
- Encode `family` (33 categories) using label encoding or target encoding.

3.6 — Sort and index:

```
df = df.sort_values(['store_nbr', 'family', 'date']).reset_index(drop=True)
```

3.7 — Save cleaned data:

```
df.to_parquet('data/processed/cleaned_data.parquet', index=False)
```

Common Mistakes to Avoid:

- ❌ Global forward-fill for oil prices across all rows without sorting by date first — this corrupts the temporal order.
- ❌ Including transferred holidays as active holidays — use `holidays[holidays['transferred'] == False]`.
- ❌ Not sorting by `(store_nbr, family, date)` before computing lag features (lags will be mismatched across groups).

PHASE 4 — Exploratory Data Analysis (EDA)

Inputs: Cleaned DataFrame.
Outputs: Minimum 10 saved figures in `/outputs/eda/`, EDA insights documented.

Steps:

4.1 — Statistical summaries:

- `.describe()` for all numerical columns.
- Per-store and per-family mean/median daily sales.
- Holiday vs. non-holiday mean sales comparison (ANOVA across national/regional/local/none; report F-statistic and p-values).
- Oil price trend over the training period (2013–2017); annotate the 2015–2016 oil price crash and its correlation with aggregate sales.

4.2 — Time-series decomposition:

```
from statsmodels.tsa.seasonal import seasonal_decompose
# Use daily data: aggregate one (store, family) series first
series = df[(df['store_nbr']==1) & (df['family']=='GROCERY I')].set_index('date')
['sales']
result = seasonal_decompose(series, model='additive', period=365)
```

- Perform for 3 representative `(store_nbr, family)` pairs (high volume: GROCERY I; promotional: BEVERAGES; volatile: PRODUCE).
- Inspect: trend direction (growth/decline 2013–2017), seasonal amplitude, residual spikes around holidays.

4.3 — Distribution analysis:

- Histogram + KDE of `sales` — confirm right-skew and zero-inflation (log1p-transform recommended).
- Box plots of `sales` by `store_type` (A–E) and by `is_national_holiday`.
- Distribution of `onpromotion` values (note zero-inflation; most family-store-days have 0 items on promotion).

4.4 — Correlation analysis:

- Pearson correlation heatmap: `sales` vs. all numerical features (`onpromotion`, `dcoilwtico`, `transactions`, lag features).
- Cross-family correlation matrix (pivot to wide format, one column per family, correlate residual sales during `onpromotion > 0` periods).
- Highlight pairs with |r| > 0.6 (potential cannibalization candidates, e.g., BEVERAGES vs. LIQUOR/WINE/BEER).

4.5 — Key visualizations to produce:

#	Plot	Purpose
1	Daily aggregate sales time-series (2013–2017)	Global trend detection + holiday spikes
2	Seasonal decomposition (3 store-family series)	Validate `period=365`, confirm annual pattern
3	Correlation heatmap (numerical features)	Feature selection signal
4	Holiday lift bar chart (national/regional/local)	Quantify multi-level holiday effect

#	Plot	Purpose
5	Sales by store type (A–E) box plot	Stratification signal
6	Oil price vs. aggregate sales (dual-axis)	Macroeconomic effect visualization
7	Cross-family correlation matrix	Cannibalization candidate identification
8	ACF/PACF plots (3 representative series)	Guide lag feature window selection
9	`onpromotion` count vs. sales scatter	Promotion-sales relationship
10	Transactions vs. sales scatter (daily)	Foot traffic as leading sales indicator

4.6 — Insights to document (minimum 3):

- Example: "*BEVERAGES and LIQUOR/WINE/BEER exhibit a negative correlation of r = −0.48 during `onpromotion > 0` weeks in Store 1, suggesting cannibalization.*"
- Example: "*The ACF plot for GROCERY I shows significant autocorrelation at lags 7, 14, and 365, confirming weekly and annual seasonality.*"
- Example: "*National holidays show a 31% mean sales lift; regional holidays show only 9% — indicating holiday effect is locale-specific and must be encoded with granularity.*"
- Example: "*Oil price dropped 60% between mid-2014 and early-2016; aggregate store sales declined ~8% over the same window, suggesting macroeconomic sensitivity.*"

Common Mistakes to Avoid:

- ❌ Computing correlations on unsorted data without time-alignment.
- ❌ Using `period=52` for daily data — correct period is **365** (or 7 for weekly seasonality; test both).
- ❌ Conflating Pearson correlation with causation in the cannibalization analysis.

PHASE 5 — Feature Engineering

Inputs: Cleaned DataFrame.
Outputs: Feature matrix saved to `/data/processed/features.parquet`.

Steps:

5.1 — Temporal features:

```
df['day_of_week']  = df['date'].dt.dayofweek
df['day_of_month'] = df['date'].dt.day
df['week_of_year'] = df['date'].dt.isocalendar().week.astype(int)
df['month']    = df['date'].dt.month
df['quarter']  = df['date'].dt.quarter
df['year']     = df['date'].dt.year
df['is_weekend'] = (df['day_of_week'] >= 5).astype(int)
```

5.2 — Holiday proximity:

```
# Pre-process holidays_events.csv (already done in Phase 3)
# Add days_to_nearest_holiday
holiday_dates = active_holidays['date'].unique()
df['days_to_nearest_holiday'] = df['date'].apply(
    lambda d: min(abs((d - h).days) for h in holiday_dates)
)
```

5.3 — Lag features (within `(Store, Dept)` group):

```
# Daily lags within (store_nbr, family) group
for lag in [1, 7, 14, 365]:
    df[f'sales_lag_{lag}d'] = df.groupby(['store_nbr', 'family'])
['sales'].shift(lag)
```

5.4 — Rolling statistics:

```
for window in [7, 14, 28]:
    df[f'rolling_mean_{window}d'] = df.groupby(['store_nbr', 'family'])
['sales'].transform(
        lambda x: x.shift(1).rolling(window).mean()
    )
    df[f'rolling_std_{window}d'] = df.groupby(['store_nbr', 'family'])
['sales'].transform(
        lambda x: x.shift(1).rolling(window).std()
    )
```

> **Critical**: Always shift by 1 before rolling to avoid target leakage.

5.5 — Promotional features:

```
# onpromotion is already in train.csv per (store_nbr, family, date)
df['onpromotion_lag_1d']   = df.groupby(['store_nbr', 'family'])
['onpromotion'].shift(1)
df['onpromotion_rolling7d'] = df.groupby(['store_nbr', 'family'])
['onpromotion'].transform(
    lambda x: x.shift(1).rolling(7).mean()
)
# Oil price lags
df['oil_lag_7d']       = df['dcoilwtico'].shift(7)  # global; no groupby needed
df['oil_rolling_28d']  = df['dcoilwtico'].shift(1).rolling(28).mean()
```

5.6 — Cannibalization proxy features:

- Identify top-3 correlated product families per store (from EDA cross-family correlation matrix during `onpromotion > 0` periods).
- For each `(store_nbr, family, date)`, add the 7-day lagged mean sales of those correlated families.

5.7 — Feature selection:

- Remove features with > 90% zero values.
- Apply Variance Inflation Factor (VIF) analysis to detect multicollinearity among numerical features.
- Drop features with VIF > 10 if using linear models.
- For tree models, collinearity is acceptable — feature selection is based on SHAP importance post-training.

5.8 — Drop NaN rows created by lags:

```
df.dropna(subset=['sales_lag_365d'], inplace=True)  # Most restrictive lag
```

Common Mistakes to Avoid:

- ❌ Computing rolling mean without `.shift(1)` — this leaks the current day's sales into itself.
- ❌ Computing lags globally without `groupby` — lags will cross `(store_nbr, family)` boundaries.
- ❌ Not dropping NaN rows from the 365-day lag (this removes the first year of data per series — expected and correct, leaving 4+ years for training).

PHASE 6 — Model Development

Inputs: Feature matrix, train/test temporal split.
Outputs: Trained model objects saved to `/models/`.

Temporal Split (non-negotiable):

```
cutoff_date = pd.Timestamp('2017-08-16')  # Official competition test start
X_train = df[df['date'] < cutoff_date].drop(columns=['sales', 'date'])
y_train = df[df['date'] < cutoff_date]['sales']
X_test  = df[df['date'] >= cutoff_date].drop(columns=['sales', 'date'])
y_test  = df[df['date'] >= cutoff_date]['sales']
```

Step 6.1 — Baseline 1: Seasonal Naive

```
# Prediction = same day's sales from 365 days ago (within each store-family
group)
y_pred_naive = df.groupby(['store_nbr','family'])['sales'].shift(365)
```

Step 6.2 — Baseline 2: Linear Regression with Gradient Descent (from scratch)

```
# Pure NumPy implementation
def gradient_descent(X, y, lr=0.001, iterations=1000):
    m, n = X.shape
    theta = np.zeros(n)
    loss_history = []
```

```
    for i in range(iterations):
        predictions = X @ theta
        errors = predictions - y
        gradient = (2/m) * (X.T @ errors)
        theta -= lr * gradient
        loss_history.append(np.mean(errors**2))
    return theta, loss_history
```

- Normalize features with `StandardScaler` before feeding to GD.
- Plot loss curve to confirm convergence.

Step 6.3 — Advanced Model 1: XGBoost

```
import xgboost as xgb
model_xgb = xgb.XGBRegressor(
    n_estimators=500, max_depth=6, learning_rate=0.05,
    subsample=0.8, colsample_bytree=0.8,
    early_stopping_rounds=50, random_state=42
)
model_xgb.fit(X_train, y_train, eval_set=[(X_test, y_test)], verbose=100)
```

Step 6.4 — Advanced Model 2: LightGBM

```
import lightgbm as lgb
model_lgb = lgb.LGBMRegressor(
    num_leaves=63, max_depth=7, learning_rate=0.05,
    n_estimators=500, min_child_samples=20,
    feature_fraction=0.8, random_state=42
)
model_lgb.fit(X_train, y_train,
              eval_set=[(X_test, y_test)],
              callbacks=[lgb.early_stopping(50)])
```

Step 6.5 — Cross-Validation (TimeSeriesSplit):

```
from sklearn.model_selection import TimeSeriesSplit, cross_val_score
tscv = TimeSeriesSplit(n_splits=5)
cv_scores = cross_val_score(model_xgb, X_train, y_train, cv=tscv,
scoring='neg_root_mean_squared_error')
```

Common Mistakes to Avoid:

- ❌ Using `train_test_split(shuffle=True)` on time-series — future data leaks into training.
- ❌ Using `KFold` instead of `TimeSeriesSplit` for cross-validation.
- ❌ Not setting `random_state=42` consistently — results will not be reproducible.

- ❌ Fitting `StandardScaler` on the entire dataset before splitting (scaler must be fit on training data only).

PHASE 7 — Model Evaluation

Inputs: Trained models, test set predictions.
Outputs: Metrics table, residual plots, SHAP analysis.

Steps:

7.1 — Compute metrics for each model:

```
from sklearn.metrics import mean_squared_error, mean_absolute_percentage_error,
r2_score
import numpy as np

def evaluate_model(y_true, y_pred, model_name):
    rmsle = np.sqrt(np.mean((np.log1p(y_pred.clip(0)) - np.log1p(y_true)) ** 2))
    rmse  = np.sqrt(mean_squared_error(y_true, y_pred))
    # MAPE: exclude zero actuals to avoid division by zero
    mask  = y_true > 0
    mape  = mean_absolute_percentage_error(y_true[mask], y_pred[mask]) * 100
    r2    = r2_score(y_true, y_pred)
    print(f"{model_name}: RMSLE={rmsle:.4f}, RMSE={rmse:.2f}, MAPE={mape:.2f}%,
R²={r2:.4f}")
    return {'model': model_name, 'RMSLE': rmsle, 'RMSE': rmse, 'MAPE': mape,
'R2': r2}
```

7.2 — Produce comparison table:

Model	RMSLE	RMSE	MAPE (%)	R^2
Seasonal Naive	—	—	—	—
GD Linear Regression	—	—	—	—
XGBoost (tuned)	—	—	—	—
LightGBM (tuned)	—	—	—	—

(Fill with actual experimental results)

7.3 — Residual analysis:

```
residuals = y_test - y_pred
plt.scatter(y_pred, residuals, alpha=0.3)
plt.axhline(0, color='red')
plt.xlabel('Predicted Sales'); plt.ylabel('Residuals')
```

- Acceptable: random scatter around zero.

- Problem indicators: funnel shape (heteroscedasticity), systematic curve (non-linearity not captured).

7.4 — SHAP analysis (on best model):

```
import shap
explainer = shap.TreeExplainer(model_xgb)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test, plot_type='bar')
```

7.5 — Bias/Variance analysis:

- **High bias (underfitting)**: Training RMSE ≈ Test RMSE, both high → increase model complexity.
- **High variance (overfitting)**: Training RMSE << Test RMSE → add regularization, reduce features, increase training data.

Common Mistakes to Avoid:

- ❌ Using MAPE as the sole metric — zero `sales` values cause division by zero; use RMSLE as primary.
- ❌ Not reporting confidence intervals or variance across CV folds.
- ❌ Evaluating on the training set and claiming good performance.

PHASE 8 — Model Finalization

Inputs: Evaluation results, best model object.
Outputs: Saved model file, final configuration JSON.

Steps:

8.1 — Select best model:

- Primary: lowest RMSE on held-out test set.
- Tiebreaker: lowest MAPE, then R^2.
- Document selection rationale explicitly.

8.2 — Save model:

```
import joblib
from datetime import datetime
model_name = f"lightgbm_{datetime.today().strftime('%Y%m%d')}_v1.pkl"
joblib.dump(model_lgb, f'models/{model_name}')
```

8.3 — Save best hyperparameters:

```
import json
best_params = model_lgb.get_params()
with open('models/best_params_lgbm.json', 'w') as f:
    json.dump(best_params, f, indent=2)
```

8.4 — Save scaler/preprocessor:

```
joblib.dump(scaler, 'models/standard_scaler.pkl')
```

8.5 — Verify reload:

```
model_reloaded = joblib.load(f'models/{model_name}')
y_pred_reloaded = model_reloaded.predict(X_test)
assert np.allclose(y_pred_reloaded, y_pred_original), "Predictions differ after
reload!"
```

8.6 — Document final configuration: Create `models/README.md` containing: model name, training date, training data date range, test RMSE/MAPE/R^2, feature list, preprocessing steps, and random seeds used.

Common Mistakes to Avoid:

- ❌ Saving model without saving the corresponding scaler/preprocessor — inference will fail.
- ❌ Not verifying that the reloaded model produces identical outputs.
- ❌ Saving the model using a path that includes spaces (cross-platform compatibility issue).

PHASE 9 — Reporting & Presentation

Inputs: All outputs from Phases 1–8, SHAP plots, cannibalization report.
Outputs: Final Jupyter Notebook, exported HTML report, slide deck (optional).

Final Report Structure:

Section	Content
1. Executive Summary	1 page: problem, approach, best result, key insight
2. Dataset Description	Schema, provenance, citation, size
3. Preprocessing	What was done, why, before/after statistics
4. EDA	Minimum 5 figures with interpretations
5. Feature Engineering	Feature table, rationale per group
6. Model Development	Pseudocode + code for each model
7. Results	Comparison table, actual vs. predicted plots
8. Cannibalization Analysis	Correlation matrix, identified pairs, lift analysis
9. Limitations	Data limitations, model assumptions, scope gaps
10. Future Work	Multi-step forecasting, SKU-level, deep learning (LSTM)

Key Visualizations for Final Report:

1. Overall pipeline flowchart (draw.io or Mermaid).
2. Model comparison bar chart (RMSE/MAPE across models).
3. Actual vs. predicted time-series for best model (top-3 store-dept pairs).
4. SHAP summary plot.
5. Cannibalization candidate heatmap.
6. Promotional lift scatter plot.

Common Mistakes to Avoid:

- ❌ Claiming model "achieves 95% accuracy" — this is a regression task; accuracy is not a valid metric.
- ❌ Omitting the Limitations section — this is a required academic element.
- ❌ Presenting results without uncertainty quantification (report CV standard deviation).
- ❌ Not running the notebook end-to-end before submission.

SECTION 4 — Semester Execution Timeline (8-Week Checklist)

```
WEEK 1: Problem Framing & Data Acquisition
[ ] Read Kaggle competition page (store-sales-time-series-forecasting) and
annotate all 6 file schemas.
[ ] Download all 6 CSV files via Kaggle API; verify file sizes and row counts.
[ ] Set up project directory structure: /data/raw/, /data/processed/,
/notebooks/, /outputs/, /models/, /reports/.
[ ] Initialize Git repository; create .gitignore (exclude /data/raw/ due to file
size ~112MB).
[ ] Write problem statement and assumptions document (1–2 pages), explicitly
noting RMSLE as primary metric.
[ ] Confirm and document the temporal train/test split: train < 2017-08-16, test
= 2017-08-16 to 2017-08-31.
[ ] DELIVERABLE: GitHub repo initialized; problem_statement.md committed.

WEEK 2: Data Preprocessing
[ ] Merge all 6 source tables (train + stores + oil + holidays + transactions).
[ ] Implement oil price forward-fill imputation; log count of filled rows.
[ ] Engineer holiday flags from holidays_events.csv; correctly handle
transferred=True rows.
[ ] Implement outlier detection; add is_outlier flag; flag 2016 earthquake date
range explicitly.
[ ] Label-encode store type; target-encode city/state and family on training data
only.
[ ] Sort data by (store_nbr, family, date).
[ ] Save cleaned data to /data/processed/cleaned_data.parquet.
[ ] DELIVERABLE: preprocessing_notebook.ipynb with documented decisions.

WEEK 3: EDA
[ ] Generate all 10 required EDA figures (see Phase 4.5 table).
[ ] Run seasonal decomposition (period=365) on 3 representative (store_nbr,
family) series.
[ ] Compute and plot cross-family correlation matrix during onpromotion>0
periods.
[ ] Run ANOVA for national/regional/local/non-holiday sales differences.
[ ] Analyze oil price trend vs. aggregate sales; annotate 2015-2016 crash.
```

```
[ ] Document minimum 4 insights as markdown cells in notebook.
[ ] DELIVERABLE: eda_notebook.ipynb + /outputs/eda/ folder committed.

WEEK 4: Feature Engineering
[ ] Implement all temporal features (day_of_week, day_of_month, week_of_year,
month, quarter, year, is_weekend).
[ ] Implement lag features (1, 7, 14, 365 days) within (store_nbr, family)
groups.
[ ] Implement rolling mean/std (7d, 14d, 28d) with proper shift(1).
[ ] Implement onpromotion lags and rolling features.
[ ] Implement oil price lags and rolling features.
[ ] Implement transactions lag features.
[ ] Implement cannibalization proxy features (top-3 correlated families per
store).
[ ] Run VIF analysis; document and remove multicollinear features for linear
model.
[ ] Drop NaN rows from 365-day lag computation; log before/after row count.
[ ] Save feature matrix to /data/processed/features.parquet.
[ ] DELIVERABLE: feature_engineering.ipynb + feature documentation table.

WEEK 5: Baseline Modeling
[ ] Implement Seasonal Naive predictor (365-day lag); compute RMSLE, RMSE, MAPE,
R² on test set.
[ ] Implement GD Linear Regression from scratch (NumPy only); apply log1p to
target before training.
[ ] Fit scaler on training data only; transform train and test.
[ ] Run GD for 1000 iterations; plot convergence curve.
[ ] Verify RMSLE for both baselines; log results.
[ ] DELIVERABLE: baseline_models.ipynb; both models saved to /models/.

WEEK 6: Advanced Modeling + Hyperparameter Tuning
[ ] Train XGBoost with default params on log1p(sales); compute test RMSLE.
[ ] Train LightGBM with default params; compute test RMSLE.
[ ] Implement TimeSeriesSplit CV with n_splits=5.
[ ] Run hyperparameter tuning for XGBoost (Optuna recommended for speed on ~3M
rows).
[ ] Run hyperparameter tuning for LightGBM.
[ ] Save best_params_xgb.json and best_params_lgbm.json.
[ ] DELIVERABLE: advanced_models.ipynb; tuned models saved.

WEEK 7: Evaluation + Cannibalization Analysis
[ ] Run full evaluation: RMSLE, RMSE, MAPE, R² for all 4+ models.
[ ] Generate model comparison table.
[ ] Generate actual vs. predicted plots for top-5 (store_nbr, family) series.
[ ] Generate residual plots; document heteroscedasticity findings.
[ ] Run SHAP analysis on best model; confirm onpromotion and lag features in top-
5.
[ ] Implement cross-family correlation analysis (residual sales, onpromotion>0
periods).
[ ] Identify and document >= 3 cannibalization pairs (Pearson r + p-value).
[ ] Compute promotional lift for >= 10 distinct onpromotion events.
[ ] DELIVERABLE: evaluation.ipynb + cannibalization_report.md.

WEEK 8: Finalization & Submission
[ ] Select and finalize best model; save with joblib.
```

```
[ ] Verify model reload produces identical predictions.
[ ] Write models/README.md with full configuration.
[ ] Write final project report (all 10 sections).
[ ] Execute full notebook end-to-end (Restart & Run All) — zero errors.
[ ] Export report to HTML/PDF.
[ ] Final Git commit with all outputs; tag as v1.0.
[ ] Prepare presentation slides (if required).
[ ] DELIVERABLE: Final submission package.
```

SECTION 5 — Academic Quality Control

5.1 Reproducibility Plan

Element	Requirement
Random seeds	Set `random_state=42` in all model constructors, all `numpy.random.seed(42)` calls, and `PYTHONHASHSEED=42` at script start.
Seed documentation	Seeds documented in models/README.md and in a `config.py` file at the project root.
Environment	`requirements.txt` or `environment.yml` committed to repo; pin all package versions (e.g., `xgboost==2.0.3`).
Data versioning	Record SHA-256 hash of all input CSVs in a `data/checksums.txt` file.
Execution order	Notebooks are numbered (01_preprocessing, 02_eda, 03_features, etc.) to enforce execution order.
Floating point	Note that floating-point results may vary across CPU architectures; report with 4 significant figures.

5.2 Version Control Usage

```
Repository structure:
retail-demand-forecasting/
├── .gitignore            # Exclude large CSV files, __pycache__,
.ipynb_checkpoints
├── README.md             # Project overview, setup instructions
├── requirements.txt      # Pinned dependencies
├── config.py             # Global seeds and constants
├── data/
│   ├── raw/              # EXCLUDED from git (listed in .gitignore)
│   └── processed/        # Parquet files - excluded if > 50MB
├── notebooks/            # Numbered execution sequence
├── outputs/
│   ├── eda/              # Saved figures
│   └── models/           # Saved model files
├── reports/              # Final report documents
```

```
└── src/                    # Reusable Python modules (preprocessing.py,
features.py, evaluate.py)
```

Git commit convention:

- `feat:` — new feature or module
- `fix:` — bug fix
- `data:` — data processing changes
- `model:` — model training or tuning changes
- `docs:` — documentation or report

Minimum required commits: At least 2 commits per week (one mid-week, one at end-of-week checkpoint).

5.3 Experiment Tracking

Use one of the following:

- **Option A (Recommended)**: MLflow with local UI.

  ```
  import mlflow
  mlflow.set_experiment("retail-demand-forecasting")
  with mlflow.start_run(run_name="xgboost_tuned_v2"):
      mlflow.log_params(best_params)
      mlflow.log_metric("rmse_test", rmse)
      mlflow.sklearn.log_model(model, "xgboost_model")
  ```

- **Option B (Lightweight)**: A manually maintained `experiments_log.csv` with columns: `run_id`, `timestamp`, `model`, `hyperparams`, `rmse_train`, `rmse_test`, `mape_test`, `notes`.

Minimum tracked: model name, hyperparameters, train RMSE, test RMSE, test MAPE, random seed, training date.

5.4 Dataset Citation

Include in all report documents and the README:

> Corporación Favorita. (2021). *Store Sales — Time Series Forecasting* [Dataset]. Kaggle Competition. Retrieved from https://www.kaggle.com/competitions/store-sales-time-series-forecasting
> License: Kaggle competition rules (for academic use).

APA format for academic report citations.

5.5 Ethical Considerations

Consideration	Status & Action
Privacy	Dataset contains no personally identifiable information (PII). ✅

Consideration	Status & Action
Bias in predictions	Verify model does not systematically under-predict for minority store types (e.g., Type C stores). Report performance stratified by store type.
Causal claims	Do NOT claim promotional markdowns *cause* sales increases — only correlation/association is demonstrated. Use precise language: "associated with," "correlated with."
Generalization claims	Do NOT claim the model will generalize to other retailers. Scope limitations must be stated explicitly.
Model deployment risks	Note that deploying inventory decisions based on model predictions without human oversight could cause financial harm if model is miscalibrated.

5.6 Plagiarism Avoidance

- All code produced must be original or properly attributed.
- If referencing external code (e.g., a SHAP tutorial), cite the source as a comment in the code cell.
- Use `git blame` history as evidence of original authorship if challenged.
- The from-scratch GD implementation must be coded independently — do NOT copy from external repositories.
- All figures and tables in the report must be generated by the project code, not from external sources.

5.7 Evaluation Methodology Correctness

Rule	Rationale
Temporal split only	Time-series data has temporal autocorrelation; random splits cause leakage.
No future features in training	All features derived from `t` must use data available strictly before time `t`.
Scaler fit on train only	Fitting on the full dataset causes test set statistics to influence training normalization (leakage).
No MAPE when actuals ≈ 0	MAPE is undefined or extreme when actual values approach zero; clip or use alternative (SMAPE, RMSE).
One test set	Do not tune on the test set. Use CV folds on training data for tuning; the held-out test set is evaluated exactly once.
Report variance	Always report CV RMSE as `mean ± std`, not just mean.

Document prepared by: AI Academic Supervisor | Last Updated: March 2026
Project: Smart Retail Demand Forecasting with Sales Decomposition and Cannibalization Analysis